

TDDMachine: Automatic Generation of the Initial Test State for Test-Driven Development Using a Large Language Model

Gleiston Guerrero-Ulloa^{1,2} , Kerly M. Triana-Arrieta¹, Efraín Díaz-Macías¹ , Carlos Rodríguez-Domínguez^{2,3} ,
and Miguel J. Hornos² 

Abstract—Writing test cases by hand is still a laborious, repetitive, and error-prone activity, and empirical evidence shows that developers test far less than they believe, treating testing as a low-value chore. Test-Driven Development (TDD), in which tests are written before production code, is nevertheless associated with 40–90% lower defect density at the cost of only a modest increase in initial development effort. This paper presents TDDMachine, a Computer-Aided Software Engineering (CASE) tool that automates the generation of the initial test state—the *Red* stage of the TDD cycle—from structured specifications (functional requirements, use cases, and user stories) using the Claude 3.5 Sonnet large language model (LLM). The requirements were derived from a systematic literature review of 109 primary studies; the technology stack (React 18, Django 4.2, PostgreSQL 15, Docker) was chosen through a weighted multi-criteria evaluation; and the generated `pytest` scripts are syntactically validated through Python abstract syntax tree analysis. The tool was evaluated with 25 participants using the System Usability Scale (SUS) and the Technology Acceptance Model (TAM), and with 10 experts who assessed test quality through a structured rubric. TDDMachine produced valid scripts in 91.4% of attempts, reached a mean expert-rated quality of 4.08/5.0, obtained a SUS score of 74.46/100 (“Good”), and exceeded the technology-acceptance threshold across all four TAM constructs. The results give empirical evidence that structured, LLM-assisted generation of the initial TDD state is both feasible and well accepted.

Index Terms—CASE tool; automated test generation; generative artificial intelligence; large language models; Test-Driven Development; `pytest`; software quality assurance; requirements engineering.

Resumen — La creación manual de casos de prueba sigue siendo una tarea laboriosa, repetitiva y propensa a errores, y la evidencia empírica muestra que los desarrolladores prueban mucho menos de lo que creen, percibiendo la escritura de pruebas como una carga de bajo valor; sin embargo, el Desarrollo Guiado por Pruebas (TDD), en el que las pruebas se escriben antes que el código de producción, se asocia con una reducción de la densidad de defectos del 40–90 % a cambio de un incremento moderado del esfuerzo inicial. Este artículo presenta TDDMachine, una herramienta CASE que automatiza la generación del estado inicial de las pruebas (la etapa *Red* del ciclo TDD) a partir de especificaciones estructuradas (requisitos funcionales, casos de uso e historias de usuario) mediante el modelo de lenguaje de gran escala (LLM) Claude 3.5 Sonnet. Los requisitos

se derivaron de una revisión sistemática de 109 estudios primarios; la pila tecnológica (React 18, Django 4.2, PostgreSQL 15, Docker) se seleccionó mediante evaluación multicriterio ponderada; y los scripts `pytest` generados se validan sintácticamente mediante análisis del árbol sintáctico abstracto de Python. La herramienta se evaluó con 25 participantes mediante los instrumentos SUS y TAM, y con 10 expertos que valoraron la calidad de las pruebas mediante una rúbrica estructurada. TDDMachine produjo scripts válidos en el 91,4 % de los intentos, alcanzó una calidad media evaluada por expertos de 4,08/5,0, obtuvo un puntaje SUS de 74,46/100 (“Bueno”) y superó el umbral de aceptación tecnológica en los cuatro constructos TAM. Los resultados aportan evidencia empírica de que la generación estructurada del estado inicial TDD asistida por LLM es viable y bien aceptada.

Palabras Clave— herramienta CASE; generación automática de pruebas; inteligencia artificial generativa; modelos de lenguaje de gran escala; Desarrollo Guiado por Pruebas; `pytest`; aseguramiento de calidad; ingeniería de requisitos.

I. INTRODUCTION

SOFTWARE testing is a cross-cutting activity of the development life cycle, responsible for verifying the functionality, stability, and reliability of code before it reaches production; yet writing tests by hand remains a costly and error-prone task, particularly in agile settings where delivery cycles are short and requirements change often [1], [2].

This burden explains a well-documented tension in professional practice: although testing is recognized as essential, developers tend to regard it as a tedious activity of little immediate value, one that competes with writing features. Field studies bear this out. The study by Beller et al., which instrumented the development environments of thousands of programmers, showed that developers spend far less time testing than they themselves estimate, and that a sizable share of development sessions goes by without a single test being run [3]. In the same vein, the survey by Straubinger and Fraser reports that a significant proportion of developers find writing tests unappealing and routinely put it off in favor of other work [4].

Evidence on project outcomes, however, points the other way once tests are written ahead of code. Test-Driven Development (TDD) inverts the traditional coding order: first a test is written that fails because the code does not yet exist (the *Red* stage), then the minimum code needed to make the test pass is implemented (the *Green* stage), and finally the code

¹School of Computer Science, Universidad Técnica Estatal de Quevedo (UTEQ), Quevedo 120501, Los Ríos, Ecuador. ²Department of Software Engineering, University of Granada, 18071 Granada, Spain. ³Research Centre for Information and Communication Technologies (CITIC), University of Granada, 18071 Granada, Spain.

Corresponding author: C. Rodríguez-Domínguez (e-mail: carlosrodriguez@ugr.es).

is refactored while the tests stay green (the *Refactor* stage) [5]. The industrial study by Nagappan et al., conducted across four teams at Microsoft and IBM, documented a drop in pre-release defect density of between 40% and 90% relative to comparable projects that did not apply TDD, at the cost of only a 15% to 35% increase in initial development time [6]. The meta-analysis by Rafique and Mišić confirms a significant improvement in external quality associated with TDD, and the controlled experiment by Erdogmus et al. on the *test-first* approach found productivity gains attributable to writing tests first [7], [8]. Taken together, these results indicate that software built from its tests is more likely to deliver the committed functionality on schedule, and that the early maintenance phase—correcting defects found after delivery—tends to be smaller than when tests are written after the code.

When in the life cycle the tests are written is therefore decisive. In the traditional *test-last* approach, tests are written once the functionality has been implemented, so they act as after-the-fact verification and inherit the design decisions already made; in the *test-first* approach that TDD promotes, by contrast, tests are written before the production code and operate as an executable specification that guides the design [5], [8]. Requirements engineering offers a natural starting point for this anticipation: when functional requirements, use cases, and user stories are documented in a structured way, following standards such as ISO/IEC/IEEE 29148, their acceptance criteria translate directly into verifiable test conditions [9]. The practical challenge is that writing those initial tests from requirements, and doing so before writing any code, demands a cognitive effort and a level of testing experience that many teams lack, which limits the adoption of TDD despite its demonstrated benefits [10].

Faced with challenges of this kind, software engineering has begun to incorporate generative artificial intelligence (AI) to automate tasks that were traditionally manual. The most recent systematic reviews document that large language models (LLMs) are being applied, with measurable success metrics, across numerous processes of the life cycle—code generation and completion, program summarization and translation, defect repair, and requirements elicitation, among others—and not only in the testing phase [11], [12]. This trend is also reflected in regional research output: *Enfoque UTE* has published reviews and studies reporting the application of machine learning and deep learning to engineering problems with quantitative performance indicators, from classification with artificial neural networks [13] to automatic bandwidth tuning in wireless networks [14] and the integration of the Internet of Things with artificial intelligence in agriculture [15]. In the specific field of testing, LLMs have shown the ability to interpret natural-language specifications and synthesize structured, executable test cases, with reported success rates that vary with the quality of the input specification [2], [16]–[20].

AI-based tools for test generation nonetheless have strengths and limitations that are worth contrasting with the proposal of this work. *ChatUniTest* [21], *TestSpark* [22], and *Penguin* [23] generate unit tests effectively, but they do so from pre-existing

source code, so they naturally operate in a *test-last* scheme and integrate into the development environment; *Pytester* [20] does translate textual descriptions into test cases, though without structured requirements management or control of the cycle state. The strength of these tools is their maturity and coverage, and their main limitation, from a TDD standpoint, is that they presuppose the existence of the very code that the methodology proposes to write *after* the test. The proposal of this work, TDDMachine, is deliberately positioned at an earlier phase of the cycle: it is a CASE (Computer-Aided Software Engineering) tool that acts exclusively on the *Red*→*Green* transition of the TDD cycle, generating the Python test script with `pytest` from structured specifications, so as to hand the development team the starting point from which to write the minimal production code. The solution was developed by deriving its requirements from a systematic literature review, selecting its technology stack (React 18, Django 4.2, PostgreSQL 15, and container deployment with Docker) through a weighted multi-criteria evaluation, and integrating the Claude 3.5 Sonnet model through a structured prompt builder and a validator that checks the syntax of the generated code against Python’s abstract syntax tree (AST). To assess the tool, two studies were designed in keeping with that scope: a usability and technology-acceptance study with 25 participants, using the SUS and TAM instruments, and a study of the quality of the generated tests with 10 experts, using a structured rubric.

This work makes three contributions. First, TDDMachine, a CASE tool that integrates in a single workflow the structured capture of three types of specification (functional requirements, use cases, and user stories; FR, UC, and US), the generation of the initial test state with an LLM, and the syntactic validation of the generated code through Python’s AST—a gap that none of the state-of-the-art tools covers in an integrated way. Second, a systematic review of 109 primary studies that characterizes the standards and notations used to structure requirements and their relationship with automatic test generation, and that grounds the design of the tool. Third, empirical evidence of the feasibility and acceptance of the proposal, obtained through a usability and technology-acceptance study with 25 participants and a test-quality study with 10 experts.

The rest of the article is organized as follows. Section II presents a systematic literature review, conducted according to Kitchenham’s guidelines, that positions the contribution against the state of the art. Section III describes the development and evaluation methodology. Section IV reports the generation, usability, acceptance, and quality results. Section V discusses the findings against the objectives and prior work and sets out the threats to validity. Finally, Section VI summarizes the conclusions and lines of future work.

II. RELATED WORK

To ground the requirements of TDDMachine and situate its contribution against the state of the art, we conducted a Systematic Literature Review (SLR) following the methodological guidelines of Kitchenham et al., which structure

TABLE I
RESEARCH QUESTIONS OF THE SYSTEMATIC REVIEW

ID	Research question
RQ1	What standards and methodologies exist for the structured capture of functional requirements?
RQ2	What features should forms and schemas have in order to standardize the capture of specifications?
RQ3	How do these standards facilitate the automatic generation of test cases?
RQ4	What existing CASE tools implement these methodologies?

TABLE II
OPERATIONALIZING THE SEARCH WITH PICOC

Element	Terms
Population (P)	Software systems, web applications, and software development projects.
Intervention (I)	Requirements-capture standards, structured forms, documentation methodologies, and CASE tools.
Comparison (C)	Traditional versus structured approaches; different documentation standards.
Outcomes (O)	Specification standardization, ease of automatic test generation, and reduced documentation time.
Context (C)	Requirements engineering, software development, and automated testing.

the process into three phases—planning, execution, and reporting [24]—and reported it in accordance with the 2020 PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) statement [25]. Before execution, we defined a protocol fixing the research questions, the search strategy and strings, the inclusion and exclusion criteria (which incorporate the assessment of methodological quality as a criterion applied at full text, EC1), and the data-extraction procedure, so as to ensure the traceability and reproducibility of the review.

A. Research Questions

The aim of the review is to determine how a software specification should be captured in a structured way to enable automatic test generation, a question that grounds the design of TDDMachine’s input forms. From this aim, we derived the four research questions in Table I: RQ1 asks about the existing standards for structuring requirements; RQ2, about the fields the capture forms should include; RQ3, about the way such structuring facilitates test generation; and RQ4, about the CASE tools that already implement these ideas and their limitations.

B. Search Strategy and Strings

The search strategy was structured with the PICOC framework (Population, Intervention, Comparison, Outcomes, Context), which operationalizes the aim of the review into five elements. Table II details the terms assigned to each element.

These five elements bound the population of interest (software projects and systems), the intervention (standards, forms, and tools for structured requirements capture), the comparison (structured versus traditional approaches), the expected outcomes (specification standardization and automatic test generation), and the context (requirements engineering and automated testing). From these terms, we built Boolean search strings specific to each type of specification—functional requirements, use cases, and user stories—which were applied over four databases (Scopus, IEEE Xplore, ACM Digital Library, and Web of Science) and complemented with a search of open-source repositories (GitHub) to identify practical implementations. Table III presents those strings in canonical form.

As Table III shows, the strings use truncation (*) and idiomatic synonyms to maximize coverage, and they were adapted to the syntax of each database—Scopus (TITLE-ABS-KEY), Web of Science (TS=), IEEE Xplore ("All Metadata"), and ACM Digital Library (AllField)—while preserving the same conceptual blocks, so that retrieval was comparable across the four sources.

C. Selection Criteria and Screening Process

Study selection was governed by the inclusion criteria (IC) and exclusion criteria (EC) in Table IV, defined a priori and each assigned to a single stage of the process to prevent duplicate application, as indicated in the *Stage* column of that table.

The process, illustrated in the PRISMA diagram of Fig. 1, comprises four consecutive filters. The first is the identification of the records retrieved by the strings of Table III in each database. The second removes duplicates (EC5), giving priority to the primary sources (ACM Digital Library and IEEE Xplore) over the bibliographic indexes (Scopus and Web of Science), which replicate much of their content. The third is eligibility, in which all the formal criteria are applied at once, over the databases’ metadata: publication year from 2015 (IC1), English language (IC6), and admissible document type—primary studies: journal articles, conference papers, book chapters, books, and doctoral theses (IC7)—while excluding non-articles—proceedings and preliminary conference material—(EC4) and secondary studies—reviews, surveys, and systematic mappings—(EC3). The fourth is title-and-abstract screening, which retains only the studies relevant to the operational aim of the review—that a structured requirements artifact be explicitly linked to the automatic generation or derivation of tests (IC2–IC5; EC2). In this way, document type is resolved once at eligibility, title-and-abstract screening is confined to thematic relevance, and the final stage—full-text reading—is reserved for the assessment of methodological quality (EC1) and the confirmation of access to the document (EC6).

Figure 1 quantifies the outcome of each filter. Starting from 6,795 records identified in the four databases, the removal of duplicates—concentrated, as expected, in Scopus and Web of Science—leaves 5,628 unique records; the eligibility criteria—year, language, and document type—reduce them to 2,642

TABLE III
SEARCH STRINGS IN CANONICAL FORM (ADAPTED TO EACH DATABASE'S SYNTAX)

Category	Search string
Functional requirements	("requirement* specification" OR "requirement* template" OR "requirement* boilerplate" OR "requirement* pattern" OR "structured requirement*" OR "controlled natural language") AND ("requirements engineering" OR "software engineering")
Use cases	("use case template" OR "use case specification" OR "use case model*" OR "use case description" OR "restricted use case" OR "structured use case") AND ("requirements engineering" OR "software engineering")
User stories	("user stor* template" OR "user stor* format" OR "user stor* pattern" OR "user stor* documentation" OR "structured user stor*") AND ("acceptance criteria" OR "INVEST" OR "agile" OR "scrum" OR "requirements engineering")

TABLE IV
INCLUSION AND EXCLUSION CRITERIA APPLIED

Inclusion criteria (IC)		Stage
IC1	Published between 2015 and the search execution date.	Elig.
IC2	Describe standards, models, or methodologies for the structured capture of requirements.	Title/abs.
IC3	Present forms, schemas, or criteria to standardize specifications.	Title/abs.
IC4	Address the relationship between requirements structuring and test generation.	Title/abs.
IC5	Deal with CASE tools for requirements management.	Title/abs.
IC6	Published in English.	Elig.
IC7	Primary studies: journal articles, conference papers, book chapters, books, or doctoral theses.	Elig.
Exclusion criteria (EC)		Stage
EC1	Purely theoretical studies without practical application, or lacking sufficient empirical evidence, methodological rigor, or reporting clarity (insufficient quality).	Full text
EC2	Outside the thematic scope: do not address structured requirements capture or its relationship with test generation.	Title/abs.
EC3	Secondary studies (systematic reviews, surveys, and mappings) that provide no primary evidence.	Elig.
EC4	Non-articles (proceedings front matter, prefaces, and preliminary conference material) and gray literature without peer review (except official standards).	Elig.
EC5	Duplicates, preliminary versions, or overlaps between categories.	Dedup.
EC6	No access to the full text.	Full text

The *Stage* column indicates the single PRISMA-flow filter (Fig. 1) at which each criterion is applied: Dedup. = deduplication; Elig. = eligibility (metadata: year, language, and document type); Title/abs. = title-and-abstract screening (thematic relevance); Full text = full-text reading (methodological quality).

(2,758 records discarded by year or language and 228 by document type); and title-and-abstract screening—incorporating

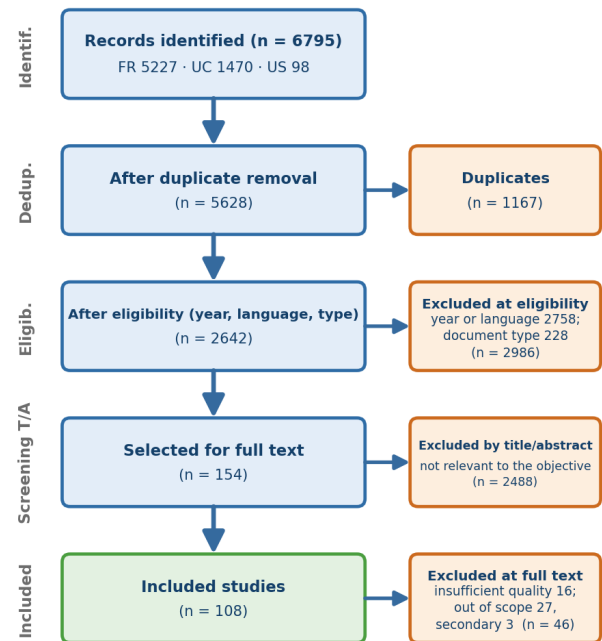


Fig. 1. Consolidated PRISMA flow of the systematic review (functional requirements, use cases, and user stories across the four databases). Eligibility applies the criteria of year, language, and document type in a single stage; title-and-abstract screening is confined to thematic relevance; and of the 155 studies read at full text, 109 were included after assessing methodological quality.

complementary studies for the user-story category, one study from the ICSR 2016 volume, and one functional-requirements study recovered after obtaining access to its full text—leaves 155 studies selected for full-text reading. These 155 were then read in full, a stage governed by the methodological-quality criterion (EC1): 109 studies were included and 46 were excluded—16 for insufficient methodological quality and 30 whose status as out of thematic scope (27) or as a secondary study (3) could only be confirmed with the full document. Table V breaks down identification, deduplication,

TABLE V
IDENTIFICATION, DEDUPLICATION, AND ELIGIBILITY BY SPECIFICATION CATEGORY AND DATABASE

Specification	Database	Identified	After duplicates	After eligibility ^b
Functional requirements	Scopus	1,646	1,312	522
	Web of Science	1,607	1,066	379
	IEEE Xplore	961	947	409
	ACM Digital Library	1,013 ^a	1,013	859
	Subtotal	5,227	4,338	2,169
Use cases	Scopus	483	360	116
	Web of Science	310	173	55
	IEEE Xplore	138	136	49
	ACM Digital Library	539	539	191
	Subtotal	1,470	1,208	411
User stories	Scopus	25	20	18
	Web of Science	18	7	4
	IEEE Xplore	6	6	4
	ACM Digital Library	49	49	36
	Subtotal	98	82	62
Total		6,795	5,628	2,642

Duplicates were removed giving priority to the primary sources (ACM, IEEE) over the indexes (Scopus, Web of Science). ^a The ACM export for functional requirements was limited to 2015–2026 (owing to the platform’s export cap), so its *Identified* column already excludes records before 2015. ^b Eligibility: all formal criteria are applied to the metadata in a single stage—publication year ≥ 2015 (IC1), English language (IC6), and admissible document type: primary studies (journal articles, conference papers, book chapters, books, and doctoral theses; IC7), excluding non-articles (EC4) and secondary studies (EC3). Of the 5,628 unique records, 2,758 were discarded by year or language and 228 by document type; title-and-abstract screening (Table VI) was applied to the 2,642 eligible records.

and eligibility by specification category and database.

The breakdown in Table V shows that functional requirements account for most of the retrieved records (5,227 of 6,795) and that the reduction by deduplication and eligibility is proportionally larger in Scopus and Web of Science, consistent with their role as indexes that largely replicate the output of the primary libraries. From the 2,642 eligible records, Table VI reports, by category, how many studies were selected for full-text reading and how many were ultimately included.

As Table VI records, of the 155 studies read at full text, 109 were included, with inclusion rates of 71% for functional requirements (87 of 122), 59% for use cases (13 of 22), and 82% for user stories (9 of 11); this last category, despite its smaller absolute volume, contributed the highest proportion of studies relevant to the aim of the review.

D. Data Extraction and Synthesis by Research Question

From each included study we extracted, using a standardized template, the following fields: year, authors, DOI, study type, application context, standards or methodologies described, specification fields identified, CASE tools mentioned, and relationship with automatic test generation. The full extraction of the 109 included studies—distributed as 87 on functional requirements, 13 on use cases, and 9 on user stories, and comprising 72 conference papers, 36 journal articles, and one doctoral thesis—is provided as supplementary material, and

TABLE VI
STUDIES SELECTED FOR FULL TEXT AND INCLUDED, BY CATEGORY

Category	Selected for full text ^a	Included ^b
Functional requirements	122	87
Use cases	22	13
User stories	11	9
Total	155	109

^a Records passing title-and-abstract screening—confined to thematic relevance with the aim of the review (IC2–IC5; EC2)—starting from the 2,642 records already eligible by year, language, and document type; includes complementary studies for the user-story category, one study from the ICSR 2016 volume, and one functional-requirements study recovered after obtaining access to its full text. ^b Studies that, after full-text reading, meet the sufficient methodological quality (EC1). Of the 155 read, 46 were excluded: 16 for insufficient methodological quality and 30 whose status as out of thematic scope (27) or as a secondary study (3) could only be confirmed with the full document.

the set is referenced in full in [26]–[134]; Fig. 2 presents the temporal distribution of this set by requirements-artifact category.

As Fig. 2 shows, output on functional requirements holds steady across the whole time window (2015–2026) and accounts for most of the corpus, while the increase observed in the most recent years coincides with the spread of large language models in software engineering. Over this same set of studies, Table VII

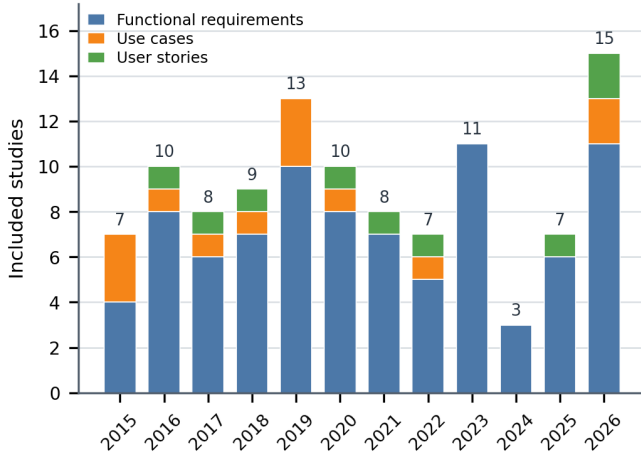


Fig. 2. Temporal distribution of the 109 included studies (2015–2026), stacked by requirements-artifact category. The figure above each bar indicates the year's total.

TABLE VII
SYNTHESIS OF EVIDENCE FROM THE 109 INCLUDED STUDIES: FREQUENCY OF STANDARDS/NOTATIONS AND OF TEST-DERIVATION TECHNIQUES (PRESENCE IN THE FULL TEXT)

Specification standard / notation	%
UML models (use case, sequence, state machine diagrams)	66
User story narrative (role-goal, 3C)	36
Gherkin / BDD (<i>Given-When-Then</i>)	33
TDD / ATDD (executable acceptance criteria)	25
Controlled natural language (CNL)	22
Use case templates (Cockburn / RUCM)	20
Acceptance criteria	20
Requirements ontologies	17
EARS (functional requirements)	10
INVEST (user stories)	6
ISO/IEC/IEEE 29148 (normative framework)	5
Test-derivation technique	%
Formalization / templates / grammars	77
Natural language processing (NLP)	44
Model-based testing (MBT)	41
Large language models (LLM)	24
Ontologies / semantic web	17

Frequencies were computed over the 109 included studies and reflect the presence of each standard, notation, or technique detected in the full-text analysis. Abbreviations: UML, *Unified Modeling Language*; BDD, *behavior-driven development*; ATDD, *acceptance test-driven development*; RUCM, *Restricted Use Case Modeling*; EARS, *Easy Approach to Requirements Syntax*. The categories are not mutually exclusive.

synthesizes the frequencies of standards, notations, and test-derivation techniques observed in the full text, whose analysis is organized below around the four research questions.

RQ1 (standards for structured capture). The evidence shows that no single normative standard governs practice; instead, each type of specification is structured through a predominant

notation. Use cases are captured mostly with UML models—use case, sequence, or state machine diagrams, present in 66% of the studies [26]–[46], [48]–[50], [52]–[54], [57]–[62], [67], [68], [70]–[72], [74]–[76], [78]–[81], [89]–[94], [97], [98], [105]–[108], [110]–[113], [118]–[122], [124]–[126], [132]–[134]—and with Cockburn-style templates or their RUCM formalization (20%) [36]–[46], [54]–[56], [58], [59], [79], [80], [100], [101], [125], [126]; functional requirements, with controlled natural language (22%) [32], [33], [40]–[43], [47]–[49], [55]–[57], [60], [61], [65]–[67], [70], [71], [106]–[108], [132], [133] and the EARS approach (10%) [27], [40], [66], [67], [70]–[72], [96], [97], [133], [134]; and user stories, with the structured role-goal narrative and the 3C scheme (36%) [26], [28], [30], [32], [33], [35], [38], [39], [41], [44], [46]–[49], [53], [54], [62]–[64], [69], [70], [72]–[75], [86]–[88], [99]–[101], [106], [107], [117], [118], [127], [128], [131], [132], the INVEST model (6%) [75], [76], [85], [86], [118], [119], and Gherkin/BDD scenarios (33%) [26]–[28], [32], [38]–[42], [45]–[47], [52], [53], [59], [60], [63], [64], [69], [70], [88]–[92], [99], [100], [106]–[108], [114], [115], [118], [119], [131], [132]. The ISO/IEC/IEEE 29148:2018 framework acts as an overarching reference, though it is explicitly cited in only a minority of studies (5%) [9], [26], [35], [46], [101], [128]. This distribution justifies TDDMachine's support for the three artifacts with their respective predominant structures.

RQ2 (form features). From the analysis of the templates described by the included studies, we identified the recurring fields that every form must capture: for functional requirements, name, description, type, acceptance criteria, and priority; for use cases, name, actors, goal, the basic flow, and the alternative flows [33], [34], [37], [40]–[47], [56]–[59], [79], [80], [98], [99], [125], [126] together with preconditions and postconditions [32], [35]–[37], [40]–[46], [50]–[53], [58], [59], [68], [69], [71], [72], [79]–[81], [83], [84], [91]–[94], [99]–[102], [116]–[119], [124]–[126], [131]–[133]; and for user stories, the structured narrative “as a [role], I want [goal], so that [benefit]” [26], [28], [30], [32], [33], [35], [38], [39], [41], [44], [46]–[49], [53], [54], [62]–[64], [69], [70], [72]–[75], [86]–[88], [99]–[101], [106], [107], [117], [118], [127], [128], [131], [132] along with their acceptance criteria [30], [35], [38], [40], [44], [46], [47], [52], [53], [69]–[71], [73], [74], [100], [101], [106], [107], [127], [128], [131], [132]. These fields, matching the canonical Cockburn templates and their RUCM formalization [36]–[46], [54]–[56], [58], [59], [79], [80], [100], [101], [125], [126], the ISO/IEC/IEEE 29148 standard, and the INVEST/Connextra model, were implemented as mandatory in the tool; those appearing in a minority of sources (status, source, dependencies, or estimation) were made optional.

RQ3 (relationship with test generation). Techniques that exploit structured or formalized input predominate: formalization through templates, grammars, or controlled natural language appears in 77% of the studies [27]–[29], [32], [33], [35]–[50], [53]–[62], [64]–[68], [70]–[72], [74]–[80], [82], [83], [85]–[93], [98]–[101], [104]–[110], [112], [113], [115]–[121], [124]–[127], [132]–[134]; natural language processing, in 44% [27], [28], [33], [37], [40], [42], [43], [45]–[50], [53]–

[55], [59], [60], [65]–[68], [71]–[73], [78]–[80], [82], [83], [86], [87], [97], [98], [103], [104], [106]–[109], [115], [116], [122], [123], [125], [126], [132], [133]; model-based testing, in 41% [27], [28], [36]–[38], [41]–[45], [47]–[50], [52], [53], [57]–[61], [67], [68], [70]–[72], [76], [77], [79], [80], [92]–[94], [104], [105], [110]–[114], [118]–[120], [131], [132]; and the use of large language models, in 24% [35], [37], [46], [47], [49], [50], [81]–[83], [85]–[87], [95]–[97], [103], [104], [108], [109], [120]–[123], [126]–[128], with a rising trend in the most recent years. Structured fields—particularly acceptance criteria and pre/postconditions—translate directly into verifiable assertions, so standardizing the input reduces the ambiguity of natural language and improves the consistency of the automatically generated tests, in line with evidence that the quality of the input specification determines that of the output [2], [16].

RQ4 (existing CASE tools). The tools identified fall into two families. The first derives tests from structured requirements specifications—for example, generators from restricted use cases (RUCM/UMTG, aToucan, MARITACA), from natural-language requirements (CiRA, TestMEReq, EARS2TF), and, in the most recent work, LLM-assisted ones (Req2Test, REST-at) [35], [37], [46], [47], [49], [50], [81]–[83], [85]–[87], [95]–[97], [103], [104], [108], [109], [120]–[123], [126]–[128]—but it usually stops at the test specification without covering the TDD cycle. The second generates tests from source code or free text—*ChatUniTest* [21], *TestSpark* [22], *Pynguin* [23], and *Pytester* [20]—so it operates at a later phase, in a *test-last* scheme. None integrates the structured capture of requirements with the LLM-assisted generation of the *Red*-stage test, a gap that TDDMachine addresses. The analysis further shows that general-purpose models with strong capabilities for synthesis and code processing—Claude 3.5 Sonnet and GPT-4—were the ones preferred in recent literature, in line with Hou *et al.*'s review of LLMs for software engineering [11].

E. Positioning Against the State of the Art

To position the contribution against a broad set of systems, Table VIII contrasts TDDMachine with fourteen representative state-of-the-art tools, grouped into three families: (i) test generators from structured requirements specifications—restricted use cases with UMTG [79], RTCM/aToucan [41], MARITACA [80], and acceptance-test generation from use cases [42]; natural-language requirements with CiRA [73], TestMEReq [117], EARS2TF [67], and requirements-to-test-case conversion [30]—; (ii) recent LLM-assisted proposals—Req2Test [47] and REST-at [109]—; and (iii) generators that start from source code or free text—*ChatUniTest* [21], *TestSpark* [22], *Pynguin* [23], and *Pytester* [20]. The comparison shows that requirements-based tools rarely integrate an LLM engine and none incorporates AST validation of the generated code or the management and export of the test cycle; LLM-based tools do not capture the three artifact types in a structured way; and code-oriented tools operate in a *test-last* scheme, once production code already exists. TDDMachine is the only proposal that brings together structured capture of the

three artifacts, an LLM engine, AST syntactic validation, and integrated management, while placing itself explicitly at the *Red* stage of the TDD cycle.

F. Threats to the Validity of the Review

The review has the usual limitations of a bounded SLR. Construct validity is conditioned by the restriction to four databases (Scopus, IEEE Xplore, ACM Digital Library, and Web of Science) and to English-language publications from 2015 onward, which may have left out relevant literature in other sources or languages; this was mitigated with search strings built on the PICOC approach—with truncation and synonyms to maximize coverage—and complemented by a search of open-source repositories. To bound the risk of selection bias, screening and extraction followed a multi-author review protocol: the second author performed the initial selection of studies; the first author reviewed the process step by step to confirm and correct the decisions; and the third, fourth, and fifth authors carried out a full review of the process. Disagreements were resolved by consensus and, failing that, by majority among the five authors. This procedure, together with the application of explicit inclusion and exclusion criteria, a standardized extraction template, and a documented record of the reasons for exclusion, mitigates single-reviewer bias and strengthens the reliability of the selection. Internal validity nonetheless retains the limitation inherent to the human interpretation of the criteria, attenuated by triangulation among reviewers.

III. METHODOLOGY

TDDMachine was developed in six successive phases—requirements analysis, specification, architectural design, module implementation, verification and validation, and deployment—under a Component-Based Software Engineering (CBSE) approach. The tool's scope was delimited explicitly: it intervenes only in the *Red*→*Green* transition of the TDD cycle, generating the test script that the development team will use as the initial state; the *Green* and *Refactor* stages fall outside its scope. Accordingly, the supported language is Python and the testing framework is `pytest`.

A. Design and Development of the Tool

The requirements were derived from the systematic review described in Section II: from the fields identified in the templates of the included studies and in the reference standards, we determined what information to capture in each form, classifying as mandatory the recurring fields—those consistently present in those templates—and as optional the rest. This analysis yielded eighteen functional requirements organized into four modules—authentication and user management, project management, specification capture, and test generation and management—and a set of non-functional requirements for security, performance, usability, integration, and availability, specified in accordance with ISO/IEC/IEEE 29148:2018 and

TABLE VIII
COMPARISON OF TDDMACHINE WITH FOURTEEN REPRESENTATIVE STATE-OF-THE-ART TOOLS

Tool	Input ^a	Artifact	Engine ^b	AST valid.	Mgmt. ^c
TDDMachine (this work)	SR	FR, UC, US	LLM	Yes	Yes
<i>Generators from structured requirements specifications</i>					
UMTG [79]	SR	UC	NLP	No	Partial
RTCM / aToucan [41]	SR	UC	NLP	No	No
Acceptance tests from UC [42]	SR	UC	NLP	No	No
MARITACA [80]	SR	UC	NLP/MBT	No	No
CiRA [73]	NL	FR	NLP/ML	No	Partial
TestMEReq [117]	SR	FR	MBT	No	No
EARS2TF [67]	SR	FR	Semiformal	No	No
Convert requirements to tests [30]	SR	FR	NLP	No	No
<i>LLM-assisted proposals</i>					
Req2Test [47]	SR	US	LLM	No	Partial
REST-at [109]	SR	FR	LLM	No	Partial
<i>Generators from source code or free text (test-last)</i>					
ChatUniTest [21]	SC	—	LLM	Partial	Partial
TestSpark [22]	SC	—	LLM/EV	Partial	Yes
Pynguin [23]	SC	—	EV	Yes	No
Pytester [20]	FT	—	RL	No	No

^a Input: SR=structured requirements; NL=natural-language requirements; SC=source code; FT=free text. ^b Engine: LLM=large language model; NLP=natural language processing; MBT=model-based testing; EV=evolutionary algorithms; RL=reinforcement learning; ML=machine learning. ^c Mgmt.=integrated review, editing, and export capabilities for the test cycle. Only TDDMachine intervenes explicitly at the *Red* stage of the TDD cycle (before production code exists). FR=functional requirements; UC=use cases; US=user stories.

ISO/IEC 25010 [9]. The technology components were selected through a weighted multi-criteria decision matrix, in which each alternative was rated from 1 to 5 on technical criteria with percentage weights; the selection resulted in React 18 with Ant Design on the frontend, Django 4.2 with Django REST Framework on the backend, PostgreSQL 15 as the database manager, and Claude 3.5 Sonnet as the generation model.

On these requirements we built a client–server architecture organized into four layers with well-defined interfaces—presentation, business logic, persistence, and infrastructure—summarized in Fig. 3. The React 18 frontend communicates with the Django backend through a REST-style (Representational State Transfer) application programming interface (API), whose interceptors attach the JWT (JSON Web Token) authentication token to every request. The business-logic layer implements three main services that realize the generation flow: `PromptBuilder`, which builds structured prompts from the type and content of each specification; `ClaudeService`, which manages the calls to the Anthropic API with automatic retries and exponential backoff on transient failures; and `PytestValidator`, which checks the syntax of the generated code by analyzing Python’s abstract syntax tree (AST). This separation isolates prompt construction, model invocation, and syntactic quality control, so that each stage can evolve or be replaced independently. To support reproducibility, `ClaudeService` invoked the model with version `claude-3-5-sonnet-XX`, temperature `YY`,

and an output-token limit of `ZZ`, over a 200,000-token context window, during the access period `WW`.

The layered organization in Fig. 3, consistent with the component-based software engineering approach adopted, makes it possible to replace or upgrade each component—for example, the language model in the business-logic layer or the database manager in the persistence layer—without altering the rest of the system.

As for the structure of the generated script, the `PromptBuilder` service instructs the model to produce `pytest` tests organized with `fixtures`—`pytest`’s mechanism for setting up and tearing down the test environment (*setup/teardown*)—and assertions derived from the acceptance criteria of the specification. One aspect specific to the *Red* stage is the handling of external dependencies of code that does not yet exist, in particular database access, LLM invocation, or network communication: the initial test state must isolate the unit under test from those dependencies through *test doubles*. In the Python ecosystem, this isolation is achieved with the standard library `unittest.mock`—which allows mock objects to be created and dependencies to be substituted through `patch`, analogous to *Mockito* in Java—and its `pytest` wrapper, the `pytest-mock` plugin (*fixture mocker*); for tests that must exercise the persistence layer, the `pytest-django` plugin provides a transactional test database through the *db fixture*. Accordingly, the `PromptBuilder` guidelines cover the generation of *fixtures* and, when the specification involves

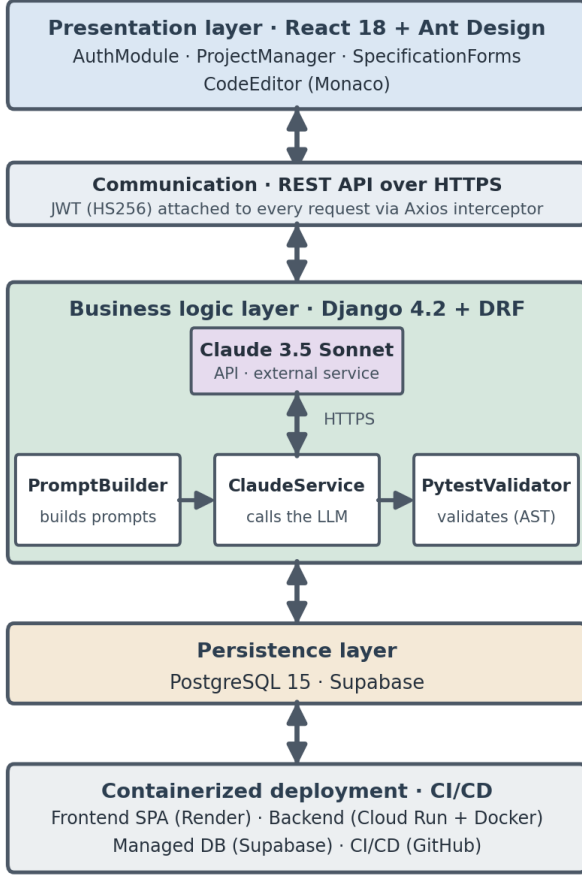


Fig. 3. Layered architecture of TDDMachine. The presentation layer (React 18 with Ant Design) captures the three specifications and reviews the script in a Monaco editor; the business-logic layer (Django 4.2 with Django REST Framework) chains the `PromptBuilder`, `ClaudeService`—the sole integration point with the Claude 3.5 Sonnet API—and `PytestValidator` services; the persistence layer (PostgreSQL 15 on Supabase) stores projects, specifications, and tests; and the infrastructure layer deploys the backend in a Docker container on Cloud Run and the frontend as a single-page application on Render.

data access, the mocking of those dependencies; strengthening the automatic generation of *mocks* and *fixtures* for complex data dependencies is one of the improvement areas identified in the expert evaluation.

The validation outcome determines how each test is handled: when it succeeds, the test is stored with the status “Generated”; on a syntax error, it is recorded with the status “Generation error” and still presented to the user for manual correction in an integrated Monaco editor with syntax highlighting, autocompletion, and real-time error detection. The management module lets users review, classify (Generated, Reviewed, Approved, Discarded), and export the approved tests as a downloadable `.py` file. Deployment was carried out on cloud infrastructure—frontend on Render, backend in a Docker container on Cloud Run, and a managed PostgreSQL database on Supabase, with continuous integration from the project’s GitHub repository.

B. Evaluation Design

The evaluation was structured into two complementary studies, designed to match the tool’s scope. Study 1, on usability and technology acceptance, involved the voluntary participation of 25 sixth-semester Software Engineering students, who completed four tasks covering the *Red*-stage flow: creating a project and registering a use case, specifying a functional requirement, writing a user story, and generating, reviewing, and exporting the `pytest` script. Usability was measured with the SUS (System Usability Scale), a standardized 10-item questionnaire on a 5-point Likert scale whose score is transformed to a 0–100 scale and interpreted according to the adjective scale of Bangor et al. [135], [136]. Technology acceptance was measured with an instrument based on the TAM (Technology Acceptance Model), comprising 16 items across four constructs—Perceived Usefulness (PU), Perceived Ease of Use (PEOU), Confidence in Results (CR), and Intention to Use (IU)—with an acceptance threshold of 3.5/5 per construct [137].

Study 2, on test quality, involved 10 testing experts who used a structured rubric, on a 1-to-5 scale, to assess five criteria—completeness, clarity, traceability, execution feasibility, and scenario coverage—over 15 generated test cases. Beyond the structural review of each script, execution feasibility was checked by running the generated tests against the production code hosted in the project’s public repositories: the experts confirmed that every script failed on its first run—the expected behavior of the *Red* stage, in which no implementation yet exists—and passed once the corresponding minimum code was added (the *Green* stage), which made it possible to verify their functional correctness and not merely their syntactic validity.

IV. RESULTS

A. Effectiveness of Automatic Generation

During the evaluation phase, 70 attempts to generate `pytest` scripts from the captured specifications were recorded. Table IX summarizes the results by specification type: the overall rate of valid scripts was 91.4%, with the best performance on functional requirements (96.4%) and the lowest on user stories (83.3%). The six failed attempts corresponded to errors detected by `PytestValidator` during AST analysis—in four cases, incomplete code blocks attributable to specifications with ambiguous acceptance criteria, and in two, structural errors in the test function; in all of them the system preserved traceability by recording the test with the status “Generation error” for manual correction.

B. Usability and Technology Acceptance

The 25 participants completed the four tasks in a mean time of 42.0 minutes (standard deviation, SD=8.2). The mean SUS score was 74.46/100 (SD=6.55; minimum 62.5; maximum 85.0), which exceeds the “Good” threshold (68 points) on the Bangor et al. scale and places the tool above the average of the systems evaluated with this instrument [136]. As for technology

TABLE IX
GENERATION OF `pytest` SCRIPTS BY SPECIFICATION TYPE

Specification	Attempts	Valid	Error	Success
Functional requirements	28	27	1	96.4%
Use cases	24	22	2	91.7%
User stories	18	15	3	83.3%
Total	70	64	6	91.4%

TABLE X
DESCRIPTIVE STATISTICS AND RELIABILITY BY TAM CONSTRUCT (N = 25)

Construct	Items	Mean	SD	α	Interp.
Perceived Usefulness (PU)	5	4.14	0.56	0.847	High
Perceived Ease of Use (PEOU)	4	4.02	0.63	0.812	High
Confidence in Results (CR)	4	3.76	0.71	0.798	Mod.–High
Intention to Use (IU)	3	4.08	0.60	0.831	High
Overall	16	4.00	0.63	0.883	High

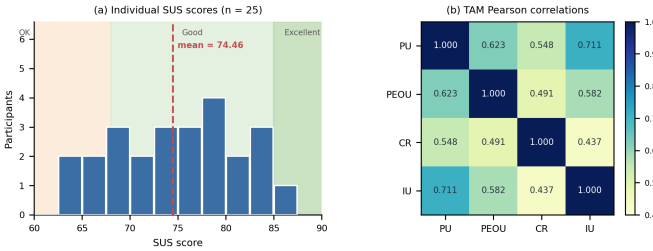


Fig. 4. Results of the usability and technology-acceptance study (n=25). (a) Distribution of individual SUS scores over the interpretive bands of the Bangor et al. scale; the dashed line marks the mean (74.46). (b) Pearson correlation matrix among the four TAM constructs; color intensity represents the magnitude of the coefficient.

acceptance, Table X presents the descriptive statistics and reliability by TAM construct.

As Table X shows, the four constructs exceeded the 3.5/5 threshold: Perceived Usefulness (4.14) and Intention to Use (4.08) obtained the highest values, while Confidence in Results (3.76) was the construct closest to the threshold, reflecting participants' uncertainty about the reliability of AI-generated code. The instrument showed adequate internal consistency, with an overall Cronbach's alpha of 0.883 and per-construct values above 0.79. The participant-level evidence is summarized in Fig. 4, which brings together the distribution of individual SUS scores (a) and the Pearson correlation matrix among the TAM constructs (b).

The distribution in Fig. 4(a) indicates that 80% of the participants (20 of 25) reached at least the "Good" threshold ($SUS \geq 68$), with no case below the minimum acceptable level of the scale. The correlation map in Fig. 4(b), in turn, shows that all associations among constructs are positive and statistically significant ($p < 0.05$), and that the strongest occurs between Perceived Usefulness and Intention to Use ($r = 0.711$; $p < 0.01$), confirming the central assumption of the TAM.

TABLE XI
TEST QUALITY BY CRITERION (1–5 SCALE, N = 10 EXPERTS)

Criterion	Mean	SD	Interpretation
Completeness	4.2	0.45	Very good
Clarity	4.3	0.42	Very good
Traceability	4.0	0.58	Good
Execution feasibility	3.9	0.67	Good
Scenario coverage	4.0	0.61	Good
Overall mean	4.08	0.55	Very good

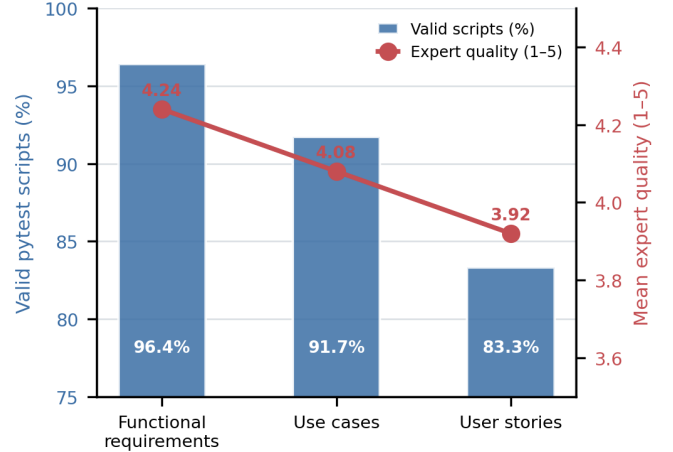


Fig. 5. Generation validity of `pytest` scripts (Table IX) and mean expert quality of the tests (Table XI) by the type of source requirements artifact. Both indicators decline in step from functional requirements toward user stories, as the ambiguity of the specification increases.

C. Quality of the Generated Tests

The expert evaluation yielded an overall mean quality of 4.08/5.0 across the five rubric criteria. Table XI presents the mean rating, standard deviation, and interpretation by criterion.

According to Table XI, the best-rated criteria were clarity (4.3) and completeness (4.2), rated "Very good," while execution feasibility (3.9) was the criterion closest to the threshold, which points to room for improvement in the precision of the assertions relative to the conditions of the source specification. Broken down by test type, the unit tests generated from functional requirements reached the highest quality (4.24), followed by integration tests from use cases (4.08) and system tests from user stories (3.92). Figure 5 relates this breakdown to the generation validity of Table IX by the type of source requirements artifact.

As Fig. 5 shows, both indicators follow a matching trend: moving from functional requirements to use cases and to user stories—that is, as the specification becomes less structured and more ambiguous—the proportion of valid scripts falls from 96.4% to 91.7% and to 83.3%, and expert quality falls from 4.24 to 4.08 and to 3.92. This concordance suggests that the degree of structure of the requirements artifact is a common determinant of both generation success and the quality of the resulting tests.

Beyond the rubric scores, execution feasibility was corroborated functionally by running the generated tests against the project’s production code: every script failed on its first run—the correct behavior of the *Red* stage, before any implementation exists—and passed after the minimum *Green*-stage code was added, which shows that the scripts are not only syntactically valid but also functionally correct as an initial test state. In the qualitative questionnaire, the experts informally compared the generated tests with those they would write by hand and agreed that the tool reproduces the tests expected for each specification and, in several cases, proposes relevant additional scenarios they had not anticipated; among the areas for improvement, they pointed to strengthening the negative and boundary-value scenarios, a recommendation taken up in the future-work lines.

V. DISCUSSION

The findings of this work are interpreted in light of the body of 109 primary studies included in the systematic review [26]–[134], which delimits the state of the art in structured requirements capture and its relationship with automatic test generation. The results provide evidence that LLM-assisted structured generation of the initial test state is feasible and well accepted. The overall valid-script rate of 91.4% falls in the upper range of what the literature reports for related tasks—for instance, the 78% of *Pytester* in generating executable test cases from natural-language text [20]—though this reference is only indicative, since it comes from different corpora, metrics, and experimental conditions rather than from a direct comparison. The agreement nonetheless suggests that combining structured specification capture with automatic AST-based syntactic validation is a promising path compared with approaches that operate on free text without integrated verification. This finding is consistent with the literature’s evidence that the quality of the input specification is the determining factor in the consistency of the generated tests: the best performance was obtained precisely on the more structured functional requirements, and the lowest on the more ambiguous user stories [2], [16].

TDDMachine’s positioning at the *Red* stage of the TDD cycle sets it apart from tools such as *ChatUniTest*, *TestSpark*, or *Penguin*, which are oriented toward generating tests from existing source code [21]–[23]. By intervening before production code exists, the tool aligns with TDD’s core principle of writing the test first and, with it, with the evidence that anticipating tests reduces defect density and the post-delivery correction effort [6]–[8]. The levels of usability (SUS = 74.46) and acceptance (all TAM constructs above the threshold, with Perceived Usefulness as the strongest predictor of Intention to Use) further indicate that the tool is perceived as useful and easy to use by users with no prior *pytest* experience, which directly addresses the TDD adoption barrier tied to the learning curve of testing [10], [136], [137]. The SUS score obtained is comparable to that of other test-driven-development support tools previously evaluated with the same instrument [138].

Several threats to validity must be considered. External validity is limited by the size and composition of the samples:

the 25 participants in Study 1 belong to a single semester and program, and the 10 experts in Study 2, though sufficient for a pilot study, are insufficient for a robust inter-rater agreement analysis; moreover, the set of specifications used does not cover every possible application domain. Generalization to professionals and industrial settings therefore requires larger-scale studies. Functional corroboration of the tests was performed by running them against the project’s own production code, so their behavior in third-party continuous-integration pipelines and over heterogeneous code bases remains to be confirmed. Finally, the measurements correspond to a first interaction session, so the stability of the perceptions under sustained use remains to be verified.

VI. CONCLUSIONS

This work presented TDDMachine, a CASE tool that automates the generation of the initial test state—the *Red* stage of the TDD cycle—from structured specifications and using the Claude 3.5 Sonnet language model, with automatic syntactic validation of the generated code through analysis of Python’s AST. The tool, whose requirements were derived from a systematic review of 109 primary studies and whose technology stack was selected by multi-criteria evaluation, produced valid *pytest* scripts in 91.4% of attempts, reached a mean expert-rated quality of 4.08/5.0, obtained a SUS score of 74.46/100, and exceeded the technology-acceptance threshold across all four TAM constructs. Together, these results validate the project’s central hypothesis: it is possible to build a CASE tool that, from structured specifications and using an LLM as the generation engine and the AST as the quality-control mechanism, automates the generation of the initial test state for the TDD methodology.

On the theoretical level, the study connects two frameworks previously treated separately: the *test-first* principle of TDD—writing the test before the code—and the technology-acceptance theory that explains the adoption of a tool from its perceived usefulness and ease of use. By placing LLM-assisted generation at the *Red* stage and showing that this anticipation is perceived as useful and usable, the work offers an empirical bridge between test anticipation as a quality practice and the acceptance of the tools that support it. The main contribution is therefore empirical and methodological: the evidence that placing a structured requirements capture ahead of LLM generation, and verifying the result through the AST, improves the proportion of valid scripts relative to approaches that operate on free text, and that this process is perceived as useful and usable by novice developers. Lines of future work include: extending the validation by actually running the scripts in controlled environments to measure the success rate at the *Green* stage; adding explicit instructions to the prompts to force negative and boundary-value scenarios, in response to the experts’ main recommendation; and broadening the evaluation to professionals and a wider diversity of domains.

ETHICS AND INFORMED CONSENT

Both evaluation studies involved voluntary participants and followed an ethics protocol. Every participant provided written informed consent before any activity; the consent form stated the voluntary nature of participation, its exclusively academic purpose, the tasks to be performed, the data to be collected, the confidentiality guarantees, the right to withdraw at any time without penalty, and the data-retention period. The collected data were handled confidentially, stored in access-restricted digital forms, and reported only in aggregate, without individual identification.

FUNDING

This research received no external funding.

CONFLICT OF INTEREST

The authors declare that they have no conflict of interest.

ARTIFICIAL INTELLIGENCE STATEMENT

The authors declare that generative artificial intelligence tools were used to assist in language editing and in the drafting and structuring of this manuscript. The core scientific content, the design of the tool, the experimental design, the data collection, and the analysis of results are the sole work and responsibility of the authors, who take full responsibility for the content of the manuscript.

DATA AND CODE AVAILABILITY

The source code of TDDMachine (frontend and backend), the deployment configuration, and the corpus of input specifications used in the evaluation are publicly available at <https://github.com/USER/tddmachine>.

REFERENCES

- [1] S. Lukaczyk, F. Kroiß, and G. Fraser, “An empirical study of automated unit test generation for Python,” *Empirical Software Engineering*, vol. 28, no. 2, p. 36, 2023.
- [2] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.
- [3] M. Beller, G. Gousios, A. Panichella, and A. Zaidman, “When, how, and why developers (do not) test in their IDEs,” in *Proc. 10th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*. New York, NY, USA: ACM, 2015, pp. 179–190.
- [4] P. Straubinger and G. Fraser, “A survey on what developers think about testing,” in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*. IEEE, 2023, pp. 80–90.
- [5] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2003.
- [6] N. Nagappan, E. M. Maximilien, T. Bhat, and L. Williams, “Realizing quality improvement through test driven development: results and experiences of four industrial teams,” *Empirical Software Engineering*, vol. 13, no. 3, pp. 289–302, 2008.
- [7] Y. Rafique and V. B. Mišić, “The effects of test-driven development on external quality and productivity: A meta-analysis,” *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 835–856, 2013.
- [8] H. Erdogmus, M. Morisio, and M. Torchiano, “On the effectiveness of the test-first approach to programming,” *IEEE Transactions on Software Engineering*, vol. 31, no. 3, pp. 226–237, 2005.
- [9] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering,” International Standard, 2018.
- [10] M. Ghafari, T. Gross, D. Fucci, and M. Felderer, “Why research on test-driven development is inconclusive?” in *Proc. 14th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas. (ESEM)*. New York, NY, USA: ACM, 2020, pp. 1–10.
- [11] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [12] A. Carleton, D. Falessi, H. Zhang, and X. Xia, “Generative AI: Redefining the future of software engineering,” *IEEE Software*, vol. 41, no. 6, pp. 34–37, 2024.
- [13] J. Molina and J. Parraga-Alava, “Artificial neural networks for classification tasks: A systematic literature review,” *Enfoque UTE*, vol. 15, no. 4, pp. 1–10, 2024.
- [14] F. Carrillo, R. T. Tandazo, and L. B. Guaman, “Analysis of artificial intelligence methods for automatic bandwidth adjustment for wireless networks,” *Enfoque UTE*, vol. 16, no. 1, pp. 45–52, 2025.
- [15] J. B. Loor and J. Parraga-Alava, “Integration of IoT and artificial intelligence in ecuadorian agriculture: A systematic literature review (2020–2025),” *Enfoque UTE*, vol. 17, no. 1, pp. 12–22, 2026.
- [16] L. Yang *et al.*, “On the evaluation of large language models in unit test generation,” in *Proc. 39th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. New York, NY, USA: ACM, 2024, pp. 1607–1619.
- [17] S. Rehan, B. Al-Bander, and A. A.-S. Ahmad, “Harnessing large language models for automated software testing: A leap towards scalable test case generation,” *Electronics*, vol. 14, no. 7, p. 1463, 2025.
- [18] S. Bhatia, T. Gandhi, D. Kumar, and P. Jalote, “Unit test generation using generative AI: A comparative performance analysis of autogeneration tools,” in *Proc. 1st Int. Workshop Large Lang. Models Code (LLM4Code)*. New York, NY, USA: ACM, 2024, pp. 54–61.
- [19] B. R. Korrapolu, P. Pinninti, and Y. R. Reddy, “Test case generation for requirements in natural language—an LLM comparison study,” in *Proc. 18th Innov. Softw. Eng. Conf. (ISEC)*. New York, NY, USA: ACM, 2025, pp. 1–5.
- [20] W. Takerngsaksiri, R. Charakorn, C. Tantithamthavorn, and Y. F. Li, “Pytester: Deep reinforcement learning for text-to-testcase generation,” *Journal of Systems and Software*, vol. 224, p. 112381, 2025.
- [21] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, “ChatUniTest: A framework for LLM-based test generation,” in *Companion Proc. 32nd ACM Int. Conf. Found. Softw. Eng. (FSE)*. New York, NY, USA: ACM, 2024, pp. 572–576.
- [22] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko, and P. Derakhshanfar, “TestSpark: IntelliJ IDEA’s ultimate test generation companion,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.: Companion (ICSE-Companion)*. New York, NY, USA: ACM, 2024, pp. 30–34.
- [23] S. Lukaczyk and G. Fraser, “Pynguin: Automated unit test generation for Python,” in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Companion (ICSE-Companion)*. New York, NY, USA: ACM, 2022, pp. 168–172.
- [24] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, “Systematic literature reviews in software engineering—a systematic literature review,” *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009.
- [25] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann *et al.*, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” p. n71, 2021.
- [26] T. N. Kudo, R. F. Bulcao-Neto, and A. M. R. Vincenzi, “A conceptual metamodel to bridging requirement patterns to test patterns,” in *33rd Brazilian Symposium on Software Engineering (SBES) / 10th Brazilian Conference on Software (CBSOFT)* 33rd Brazilian Symposium on Software Engineering (SBES) / 10th Brazilian Conference on Software (CBSOFT), 2019, pp. 155.0–160.0.
- [27] S. Sandhu, S. Rajendran, F. Sarracini, and M. Soleimani, “A framework for integrated behaviour driven development and model based feature design and verification,” in *International Conference on Vehicle Technology and Intelligent Transport Systems, VEHITS - Proceedings*. Science and Technology Publications, Lda, 2026, pp. 439 – 445.
- [28] H. Bünder and H. Kuchen, “A model-driven approach for behavior-driven gui testing,” in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1742–1751.
- [29] A. Agrawal, B. Zhang, Y. Shivalingaiah, M. Vierhauser, and J. Cleland-Huang, “A requirements-driven platform for validating field operations of small uncrewed aerial vehicles,” in *2023 IEEE 31st International Requirements Engineering Conference (RE)*, 2023, pp. 29–40.
- [30] A. Gupta and P. Bera, “A software tool to convert requirements to test cases,” in *Proceedings of the 6th International Workshop on Requirements Engineering and Testing*, ser. RET ’19. IEEE Press, 2019, p. 9–12.
- [31] P. Mahanto, S. K. Barisal, and D. P. Mohapatra, “Achieving mc/dc using uml communication diagram,” in *Proceedings - 2018 International Conference on Information Technology, ICIT 2018*. Institute of Electrical and Electronics Engineers Inc., 2018, pp. 73 – 78.
- [32] T. N. Kudo, R. d. F. Bulcão-Neto, V. V. G. Neto, and A. M. R. Vincenzi, “Aligning requirements and testing through metamodeling and patterns:

- design and evaluation,” *Requirements Engineering*, vol. 28, no. 1, pp. 97 – 115, 2023.
- [33] A. Intana, K. Tantayakul, K. Laosen, and S. Charoenreh, “An approach of test case generation with software requirement ontology,” *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 8, pp. 1005 – 1014, 2023.
- [34] A. Intana, K. Laosen, and T. Sriraksa, “An automated impact analysis approach for test cases based on changes of use case based requirement specifications,” *INTERNATIONAL JOURNAL OF ADVANCED COMPUTER SCIENCE AND APPLICATIONS*, vol. 14.0, no. 1.0, pp. 967.0–980.0, 2023.
- [35] S. Orekhov, P. Taran, and N. Bahatskyi, “An example of developing a high-level requirements specification for ai-based software using chatgpt,” *International Journal of Wireless and Microwave Technologies*, vol. 16, no. 3, pp. 111 – 127, 2026.
- [36] M. L. Roldán, M. Vegetti, S. Gonnet, H. Leone, and M. Marciszack, “An ontology for specifying and tracing requirements engineering artifacts and test artifacts,” *CLEI Electronic Journal (CLEIej)*, vol. 22, no. 1, 2019.
- [37] T.-B. Trinh, T.-V. Nguyen, N.-M. Le, and N. V. Ha, “Fstg: Few-shot test case generation from use case specifications using large language models,” *Communications in Computer and Information Science*, vol. 2945 CCIS, pp. 302 – 316, 2026.
- [38] A. C. R. Paiva, D. Maciel, and A. R. da Silva, “From requirements to automated acceptance tests with the rsl language,” in *14th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE) 14th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*, vol. 1172.0, 2020, pp. 39.0–57.0.
- [39] A. C. Oran, E. Nascimento, G. Santos, and T. Conte, “Analysing requirements communication using use case specification and user stories,” in *Proceedings of the XXXI Brazilian Symposium on Software Engineering*, ser. SBES ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 214–223.
- [40] A. Veizaga, M. Alferez, D. Torre, M. Sabetzadeh, L. Briand, and E. Pitskhelauri, “Leveraging natural-language requirements for deriving better acceptance criteria from models,” in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 218–228.
- [41] T. Yue, S. Ali, and M. Zhang, “Rtcm: a natural language based, automated, and practical test case generation framework,” in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ser. ISSTA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 397–408.
- [42] C. Wang, F. Pastore, A. Goknil, and L. C. Briand, “Automatic generation of acceptance test cases from use case specifications: An nlp-based approach,” *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 585–616, 2022.
- [43] M. Zhang, T. Yue, S. Ali, B. Selic, O. Okariz, R. Norgre, and K. Intxausti, “Specifying uncertainty in use case models,” *Journal of Systems and Software*, vol. 144, pp. 573 – 603, 2018.
- [44] S. Khamaisheh and D. Xu, “Software security testing via misuse case modeling,” in *15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech) 15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)*, 2017, pp. 534.0–541.0.
- [45] D. Clerissi, M. Leotta, G. Reggio, and F. Ricca, “Towards the generation of end-to-end web test scripts from requirements specifications,” in *2017 IEEE 25th International Requirements Engineering Conference Workshops (REW)*, 2017, pp. 343–350.
- [46] H. Li, L. Zheng, and Q. Cai, “A test-driven approach for refining use case specifications of software requirements with llms,” *Lecture Notes in Computer Science*, vol. 16229 LNCS, pp. 189 – 208, 2026.
- [47] M. França and F. Benitti, “Req2test: An approach for the automatic generation of unit tests from user stories using llms,” in *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1654–1663.
- [48] J. Miranda, A. C. R. Paiva, and A. R. da Silva, “Preliminary experiences in requirements-based security testing,” *Communications in Computer and Information Science*, vol. 1266 CCIS, pp. 412 – 425, 2020.
- [49] N. Bhatia, J. Singh, and Vandana, “Automated test case generation from unstructured software requirements using advanced nlp techniques,” in *2025 12th International Conference on Reliability, Infocom Technologies and Optimization, Trends and Future Directions, ICRITO 2025*. Institute of Electrical and Electronics Engineers Inc., 2025.
- [50] W. Jang and R. Young Chul Kim, “Automatic test case generation mechanism with natural language-based korean requirement specifications,” *IEEE Access*, vol. 13, pp. 177 305 – 177 317, 2025.
- [51] D. H. Longo, P. Vilain, L. P. da Silva, and R. d. S. Mello, “A web framework for test automation: user scenarios through user interaction diagrams,” in *Proceedings of the 18th International Conference on Information Integration and Web-Based Applications and Services*, ser. iiWAS ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 458–467.
- [52] S. Tatale and V. Chandra Prakash, “Enhancing acceptance test driven development model with combinatorial logic,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 10, pp. 268 – 278, 2020.
- [53] T. Rocha Silva, M. Winckler, and H. Trætteberg, “Ensuring the consistency between user requirements and task models: A behavior-based automated approach,” *Proc. ACM Hum.-Comput. Interact.*, vol. 4, no. EICS, 2020.
- [54] C. Gralhã, R. Pereira, M. Goulao, and J. Araujo, “On the impact of using different templates on creating and understanding user stories,” in *Proceedings of the IEEE International Conference on Requirements Engineering*. IEEE Computer Society, 2021, pp. 209 – 220.
- [55] I. L. Araújo, I. S. Santos, J. B. F. Filho, R. M. Andrade, and P. S. Neto, “Generating test cases and procedures from use cases in dynamic software product lines,” in *Proceedings of the ACM Symposium on Applied Computing*. Association for Computing Machinery, 2017, pp. 1296 – 1301.
- [56] A. C. Corrêa Souza, F. L. S. Nunes, and M. E. Delamaro, “An automated functional testing approach for virtual reality applications,” *Software Testing Verification and Reliability*, vol. 28, no. 8, 2018.
- [57] S. Nogueira, H. L. S. Araujo, R. B. S. Araujo, J. Iyoda, and A. Sampaio, “Automatic generation of test cases and test purposes from natural language,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9526, pp. 145 – 161, 2016.
- [58] S. Tiwari and A. Gupta, “An approach of generating test requirements for agile software development,” in *ACM International Conference Proceeding Series*, vol. 18-20-February-2015. Association for Computing Machinery, 2015, pp. 186 – 195.
- [59] C. Wang, F. Pastore, A. Goknil, L. Briand, and Z. Iqbal, “Automatic generation of system test cases from use case specifications,” in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ser. ISSTA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 385–396.
- [60] J. Baxter, G. Carvalho, A. Cavalcanti, and F. R. Júnior, “Roboworld: Verification of robotic systems with environment in the loop,” *Form. Asp. Comput.*, vol. 35, no. 4, 2023.
- [61] G. Carvalho, A. Cavalcanti, and A. Sampaio, “Modelling timed reactive systems from natural-language requirements,” *Form. Asp. Comput.*, vol. 28, no. 5, p. 725–765, 2016.
- [62] M. Ecar, F. Kepler, and J. P. S. da Silva, “Cosmic user story standard,” in *19th International Conference on Agile Software Development (XP) 19th International Conference on Agile Software Development (XP)*, vol. 314.0, 2018, pp. 3.0–18.0.
- [63] J. Saraiva and S. Soares, “Adoption of the lgpd inventory in the user stories and bdd scenarios creation,” in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*, ser. SBES ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 416–421.
- [64] J. Matula and F. Hunka, “Enterprise ontology-driven development,” *Lecture Notes in Business Information Processing*, vol. 332, pp. 3 – 15, 2018.
- [65] S. Charoenreh and A. Intana, “Enhancing software testing with ontology engineering approach,” in *2019 23rd International Computer Science and Engineering Conference (ICSEC)*, 2020, pp. 186–191.
- [66] J. W. Lim, T. K. Chiew, M. T. Su, S. Ong, H. Subramaniam, M. B. Mustafa, and Y. K. Chiam, “Test case information extraction

- from requirements specifications using NLP-based unified boilerplate approach,” *Journal of Systems and Software*, vol. 211, p. 112005, 2024.
- [67] H. Liu, Y. Li, and Z. Li, “Ears2tf: A tool for automated planning test from semi-formalized requirements,” in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*. Knowledge Systems Institute Graduate School, 2022, pp. 469 – 470.
 - [68] H. Zheng, J. Feng, W. Miao, and G. Pu, “Generating test cases from requirements: A case study in railway control system domain,” in *Proceedings - 2021 International Symposium on Theoretical Aspects of Software Engineering, TASE 2021*. Institute of Electrical and Electronics Engineers Inc., 2021, pp. 183 – 190.
 - [69] M. Nguyen, N. Hochgeschwender, and S. Wrede, “An analysis of behaviour-driven requirement specification for robotic competitions,” in *IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE)/IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE)*, 2023, pp. 17.0–24.0.
 - [70] J. Fischbach, M. Junker, A. Vogelsang, and D. Freudenstein, “Automated generation of test models from semi-structured requirements,” in *Proceedings - 2019 IEEE 27th International Requirements Engineering Conference Workshops, REW 2019*. Institute of Electrical and Electronics Engineers Inc., 2019, pp. 263 – 269.
 - [71] O. Olajubu, “Automated test case generation from domain-specific high-level requirement models,” *Proceedings of the 2015 Conference on Research in Adaptive and Convergent Systems (RACS)*, 2018.
 - [72] F. Pudlitz, F. Brokhausen, and A. Vogelsang, “What am i testing and where? comparing testing procedures based on lightweight requirements annotations,” *EMPIRICAL SOFTWARE ENGINEERING*, vol. 25.0, no. 4.0, pp. 2809.0–2843.0, 2020.
 - [73] J. Frattini, J. Fischbach, and A. Bauer, “Cira: An open-source python package for automated generation of test case descriptions from natural language requirements,” in *31st IEEE International Requirements Engineering Conference (RE)/31st IEEE International Requirements Engineering Conference (RE)*, 2023, pp. 68.0–71.0.
 - [74] G. Holodnik-Janczura, “The extension of user story template structure with an assessment question based on the kano model,” in *37th International Conference on Information Systems Architecture and Technology (ISAT)/37th International Conference on Information Systems Architecture and Technology (ISAT)*, vol. 523.0, 2017, pp. 137.0–150.0.
 - [75] G. Lucassen, F. Dalpiaz, E. M. van der Werf, and S. Brinkkemper, “The use and effectiveness of user stories in practice,” in *22nd International Working Conference on Requirements Engineering - Foundation for Software Quality/22nd International Working Conference on Requirements Engineering - Foundation for Software Quality*, vol. 9619.0, 2016, pp. 205.0–222.0.
 - [76] W. Abdeen, X. Chen, and M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *REQUIREMENTS ENGINEERING*, vol. 28.0, no. 1.0, pp. 117.0–144.0, 2023.
 - [77] R. Venkatesh, U. Shrotri, A. Zare, and S. Agrawal, “Cost-effective functional testing of reactive software,” in *2015 International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*, 2016, pp. 67–77.
 - [78] A. Ansari, M. B. Shagufta, A. Sadaf Fatima, and S. Tehreem, “Constructing test cases using natural language processing,” in *Proceedings of the 3rd IEEE International Conference on Advances in Electrical and Electronics, Information, Communication and Bio-Informatics, AEEICB 2017*. Institute of Electrical and Electronics Engineers Inc., 2017, pp. 95 – 99.
 - [79] C. Wang, F. Pastore, A. Goknil, L. C. Briand, and Z. Iqbal, “Umtg: A toolset to automatically generate system test cases from use case specifications,” in *2015 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ESEC/FSE 2015 - Proceedings*. Association for Computing Machinery, Inc, 2015, pp. 942 – 945.
 - [80] L. Erazo, E. Martins, and J. G. Gregi, “Maritaca: from textual use case descriptions to behavior models,” in *47th IEEE/IFIP Annual International Conference on Dependable Systems and Networks (DSN)/47th IEEE/IFIP Annual International Conference on Dependable Systems and Networks (DSN)*, 2017, pp. 83.0–90.0.
 - [81] K. Louzaoui and K. Benlhachmi, “Invalid input data and conformity testing for an object-oriented specification,” in *2016 International Conference on Information Technology for Organizations Development, IT4OD 2016*. Institute of Electrical and Electronics Engineers Inc., 2016.
 - [82] M. Borg, J. Bengtsson, H. Österling, A. Hagelborn, I. Gagner, and P. Tomaszewski, “Quality assurance of generative dialog models in an evolving conversational agent used for swedish language practice,” in *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI*, ser. CAIN ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 22–32.
 - [83] J. Wen, L. Wu, X. Chen, X. Zhang, and X. Ma, “Research on formal specification generation methods for airborne software testing,” in *Proceedings of the 2026 6th International Conference on Computer Network Security and Software Engineering*, ser. CNSSE ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 184–189.
 - [84] S. Liu, “Validating formal specifications using testing-based specification animation,” in *Proceedings of the 4th FME Workshop on Formal Methods in Software Engineering*, ser. FormalISE ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 29–35.
 - [85] R. Khaldi, A. Lehmann, B. Ghita, and U. Trick, “Agentic-ai framework for integrated design, implementation, testing, and operation of digital twin networks,” *IEEE Open Journal of the Communications Society*, vol. 7, pp. 4352–4375, 2026.
 - [86] G. B. Herwanto, W. L. Muhamad, and W. Maaraja, “Enhancing automated user story quality assessment with large language models,” in *Proceedings of 2025 IEEE International Conference on Data and Software Engineering, ICoDSE 2025*. Institute of Electrical and Electronics Engineers Inc., 2025, pp. 196 – 201.
 - [87] C. Wu, C. Wang, T. Li, and Y. Zhai, “A node-merging based approach for generating istar models from user stories,” in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*. Knowledge Systems Institute Graduate School, 2022, pp. 257 – 262.
 - [88] C. A. Martins, C. Dorneles, A. Barisic, T. R. Silva, and M. Winckler, “Cm-dir: A method to support the specification of the user’s dynamic behavior in recommender systems,” in *10th IFIP WG 13.2 International Working Conference on Human-Centered Software Engineering (HCSE)/10th IFIP WG 13.2 International Working Conference on Human-Centered Software Engineering (HCSE)*, vol. 14793.0, 2024, pp. 26.0–46.0.
 - [89] C. Wiecher, J. Greenyer, and J. Korte, “Test-driven scenario specification of automotive software components,” in *ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)/ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2019, pp. 12.0–17.0.
 - [90] C. Wiecher and J. Greenyer, “Besos: A tool for behavior-driven and scenario-based requirements modeling for systems of systems,” in *CEUR Workshop Proceedings*, vol. 2857. CEUR-WS, 2021.
 - [91] C. Wiecher, S. Japs, L. Kaiser, J. Greenyer, R. Dumitrescu, and C. Wolff, “Scenarios in the loop: Integrated requirements analysis and automotive system validation,” in *Proceedings - 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS-C 2020 - Companion Proceedings*. Association for Computing Machinery, Inc, 2020, pp. 199 – 208.
 - [92] M. N. Zafar, W. Afzal, and E. Enoiu, “Towards a workflow for model-based testing of embedded systems,” in *12th International Workshop on Automating TEST Case Design, Selection, and Evaluation (A-TEST)/12th International Workshop on Automating TEST Case Design, Selection, and Evaluation (A-TEST)*, 2021, pp. 33.0–40.0.
 - [93] S. Liu and S. Nakajima, “Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing,” *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 691–712, 2022.
 - [94] T. Diniz, E. L. Alves, A. G. Silva, and W. L. Andrade, “Reducing the discard of mbt test cases using distance functions,” in *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, ser. SBES ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 337–346.
 - [95] Y. Shimizu and K. Ueda, “Automatic test cases generation method by structuring specification documents using seq2seq,” in *2025 1st International Conference on Consumer Technology, ICCT-Pacific 2025*. Institute of Electrical and Electronics Engineers Inc., 2025.
 - [96] M. E. Salari, “Test automation for industrial programmable logic controller software: From requirements to executable tests,” Ph.D. dissertation, Mälardalen University, Västerås, Sweden, 2026.
 - [97] B. Wei, “Requirements are all you need: From requirements to code with llms,” in *32nd IEEE International Requirements Engineering Conference*

- (RE)32nd IEEE International Requirements Engineering Conference (RE), 2024, pp. 416.0–422.0.
- [98] G. K. Palshikar, N. Ramrakhiyani, S. Patil, S. Pawar, S. Hingmire, V. Varma, and P. Bhattacharyya, “Extraction of message sequence charts from software use-case descriptions,” in *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, vol. 2. Association for Computational Linguistics (ACL), 2019, pp. 130 – 137.
- [99] J. Saraiva and S. Soares, “Privacy and security documents for agile software engineering: An experiment of Ispd inventory adoption,” in *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2023, pp. 1–9.
- [100] S. Rodriguez, J. Thangarajah, and M. Winikoff, “A behaviour-driven approach for testing requirements via user and system stories in agent systems,” in *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS ’23. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2023, p. 1182–1190.
- [101] J. Medeiros, A. Vasconcelos, C. Silva, and M. Goulão, “Requirements specification for developers in agile projects: Evaluation by two industrial case studies,” *Information and Software Technology*, vol. 117, 2019.
- [102] G. Fisher and C. Johnson, “Making formal methods more relevant to software engineering students via automated test generation,” in *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, ser. ITiCSE ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 224–229.
- [103] A. Fathima, K. Indulekha, and Shweta, “Automated detection and mitigation of requirement bad smells in software requirement specifications,” *Learning and Analytics in Intelligent Systems*, vol. 63, pp. 119 – 128, 2026.
- [104] S. Morales, R. Clarisó, and J. Cabot, “A dsl for testing llms for fairness and bias,” in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 203–213.
- [105] T. Bandyszak, M. Rzepka, T. Weyer, and K. Pohl, “Supporting the validation of structured analysis specifications in the engineering of information systems by test path exploration,” in *ICEIS 2015 - 17th International Conference on Enterprise Information Systems, Proceedings*, vol. 2. SciTePress, 2015, pp. 252 – 259.
- [106] T. R. Silva and B. Fitzgerald, “Empirical findings on bdd story parsing to support consistency assurance between requirements and artifacts,” in *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 266–271.
- [107] T. Rocha Silva, “Towards a domain-specific language to specify interaction scenarios for web-based graphical user interfaces,” in *Companion of the 2022 ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, ser. EICS ’22 Companion. New York, NY, USA: Association for Computing Machinery, 2022, p. 48–53.
- [108] F. Arruda, F. Barros, and A. Sampaio, “Deriving sound test scripts from requirements written in a controlled natural language,” *Lecture Notes in Computer Science*, vol. 16363 LNCS, pp. 101 – 118, 2026.
- [109] N. Leon-Quinstedt, B. Lindgren, M. Yurdakul, and F. G. d. O. Neto, “Rest-at: An llm-based tool for automating traceability between requirements and test cases,” in *Proceedings of the 7th ACM/IEEE International Conference on Automation of Software Test*, ser. AST ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1–11.
- [110] W. S. Jang and R. Y. C. Kim, “Automatic generation mechanism of cause-effect graph with informal requirement specification based on the korean language,” *Applied Sciences (Switzerland)*, vol. 11, no. 24, 2021.
- [111] Z. A. Hamza and M. Hammad, “Generating test sequences from uml use case diagram: A case study,” in *2020 Second International Sustainability and Resilience Conference: Technology and Innovation in Building Designs(51154)*, 2020, pp. 1–6.
- [112] E. B. dos Santos, L. S. a. da Costa, B. S. Aragão, I. de Sousa Santos, and R. M. de Castro Andrade, “Extraction of test cases procedures from textual use cases to reduce test effort: Test factory experience report,” in *Proceedings of the XVIII Brazilian Symposium on Software Quality*, ser. SBQS ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 266–275.
- [113] E. M. Fredericks and B. H. C. Cheng, “Automated generation of adaptive test plans for self-adaptive systems,” in *Proceedings of the 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, ser. SEAMS ’15. IEEE Press, 2015, p. 157–168.
- [114] R. Fazzolino and G. N. Rodrigues, “Feature-trace: Generating operational profile and supporting testing prioritization from bdd features,” in *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, ser. SBES ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 332–336.
- [115] L. Campanile, M. S. d. Biase, S. Marrone, M. Raimondo, and L. Verde, “On the evaluation of bdd requirements with text-based metrics: The etcs-13 case study,” *Smart Innovation, Systems and Technologies*, vol. 309, pp. 561 – 571, 2022.
- [116] M. Motwani and Y. Brun, “Automatically generating precise oracles from structured natural language specifications,” in *Proceedings of the 41st International Conference on Software Engineering*, ser. ICSE ’19. IEEE Press, 2019, p. 188–199.
- [117] N. A. Moketar, M. Kamalrudin, S. Sidek, M. Robinson, and J. Grundy, “Testmereg: generating abstract tests for requirements validation,” in *Proceedings of the 3rd International Workshop on Software Engineering Research and Industrial Practice*, ser. SER&IP ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 39–45.
- [118] D. N. Jorge, P. D. L. Machado, E. L. G. Alves, and W. L. Andrade, “Integrating requirements specification and model-based testing in agile development,” in *26th IEEE International Requirements Engineering Conference (RE)26th IEEE International Requirements Engineering Conference (RE)*, 2018, pp. 336.0–346.0.
- [119] M. N. Zafar, W. Afzal, E. Enoui, A. Stratis, A. Arrieta, and G. Sagardui, “Model-based testing in practice: An industrial case study using graphwalker,” in *Proceedings of the 14th Innovations in Software Engineering Conference (Formerly Known as India Software Engineering Conference)*, ser. ISEC ’21. New York, NY, USA: Association for Computing Machinery, 2021.
- [120] S. Varpe, U. Mittal, R. K. Bhatia, and Ashpreet, “User acceptance test case generation using llms,” in *2025 IEEE International Conference on Collaborative Advances in Software and Computing (CASCON)*, 2025, pp. 532–538.
- [121] R. Sinha, S. Patil, C. Pang, V. Vyatkin, and B. Dowdeswell, “Requirements engineering of industrial automation systems: Adapting the cesar requirements meta model for safety-critical smart grid software,” in *IECON 2015 - 41st Annual Conference of the IEEE Industrial Electronics Society*. Institute of Electrical and Electronics Engineers Inc., 2015, pp. 2172 – 2177.
- [122] T. Ishinari, K. Ueda, and Y. Shimizu, “Improving training data quality for automated test case generation using machine learning and its evaluation,” in *CIIS 2025 - 2025 the 8th International Conference on Computational Intelligence and Intelligent Systems*. Association for Computing Machinery, Inc, 2025, pp. 44 – 51.
- [123] T. Ishinari and K. Ueda, “Data augmentation method for improving the accuracy of automated test case generation using attention seq2seq model,” in *2026 IEEE 2nd International Conference on Consumer Technology, ICCT-Pacific 2026*. Institute of Electrical and Electronics Engineers Inc., 2026, pp. 490 – 493.
- [124] I. Chernorutsky, P. Drobintsev, and V. Kotlyarov, “Industrial approach in requirement engineering,” in *CEUR Workshop Proceedings*, vol. 1989. CEUR-WS, 2018, pp. 388 – 400.
- [125] P. X. Mai, F. Pastore, A. Goknil, and L. C. Briand, “Mcp: A security testing tool driven by requirements,” in *Proceedings - 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion, ICSE-Companion 2019*. Institute of Electrical and Electronics Engineers Inc., 2019, pp. 55 – 58.
- [126] T.-B. Trinh and N.-T. Truong, “Tcg: an approach to improving test-driven development by exploiting use case specifications,” *International Journal of System Assurance Engineering and Management*, vol. 17, no. 4, pp. 1159 – 1174, 2026.
- [127] E. Dilozenzo, F. Ramos, D. Albuquerque, M. Perkusich, H. Almeida, and A. Perkusich, “Recommending test cases in agile software development with a taxonomy-guided knn approach,” in *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1708–1716.
- [128] P. Baker, B. Manu, R. Moussa, and F. Sarro, “Industrial deployment of an ai multi-agent system for requirements-driven code verification,”

- in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 485–495.
- [129] T. Yue, H. Zhang, S. Ali, and C. Liu, “A practical use case modeling approach to specify crosscutting concerns,” in *Software Reuse: Bridging with Social-Awareness (ICSR 2016)*, LNCS, ser. Lecture Notes in Computer Science, vol. 9679. Cham: Springer, 2016, pp. 89–105.
 - [130] H.-V. Tran, N. P. Chi, and P. N. Hung, “Famres: A framework for automated testing of microservices based on requirement specifications,” *Lecture Notes in Networks and Systems*, vol. 1596 LNNS, pp. 356 – 366, 2025.
 - [131] E. C. dos Santos and P. Vilain, “Automated acceptance tests as software requirements: An experiment to compare the applicability of fit tables and gherkin language,” in *19th International Conference on Agile Software Development (XP)19th International Conference on Agile Software Development (XP)*, vol. 314.0, 2018, pp. 104.0–119.0.
 - [132] M. I. Malik, M. A. Sindhu, A. S. Khattak, R. A. Abbasi, and K. Saleem, “Automating test oracles from restricted natural language agile requirements,” *Expert Systems*, vol. 38, no. 1, 2020.
 - [133] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy, and A. Ibrahim, “Srcm: A semi formal requirements representation model enabling system visualisation and quality checking,” in *9th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)9th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, 2021, pp. 278.0–285.0.
 - [134] A. Crapo, A. Moitra, C. McMillan, and D. Russell, “Requirements capture and analysis in assert(tm),” in *Proceedings - 2017 IEEE 25th International Requirements Engineering Conference, RE 2017*. Institute of Electrical and Electronics Engineers Inc., 2017, pp. 283 – 291.
 - [135] J. Brooke, “SUS: A “quick and dirty” usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.
 - [136] A. Bangor, P. Kortum, and J. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
 - [137] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.
 - [138] G. Guerrero-Ulloa, D. Carvajal-Suárez, P. Novais, M. J. Hornos, and C. Rodríguez-Domínguez, “Test-driven development tool for IoT systems,” *IEEE Software*, vol. 42, no. 4, pp. 58–67, 2025.